list wraps over to the beginning of the list. You are given an arbitrary pointer to an element in the list. You need to find the minimum element in the list. You can make the simplifying assumption that all the elements are distinct.

Finding the minimum in a circular sorted linked list

The simplest approach is to use linear search. Start traversing the list from the pointer you have been given. If you reach a node whose value is smaller than the previous node value, then you have reached the minimum. Given below is the pseudo-code for this solution:

```
struct node* find_min(struct node* p)
{
    struct node* curr = p;
    struct node* prev = NULL;

    do
    {
        If (prev != NULL)
        {
            prev_val = prev->value;
            curr_val = curr->value;
            if (curr_val < prev_value) //we have found the minimum
                return curr;
        }
        prev = curr ;
        curr = curr->next;
    } while (curr->next != p);

    return p;
  }
}
```

Complexity of the simple solution

What is the time complexity of the simple solution? Since we are doing a linear search on the linked list, it is O(n)—where n is the number of elements on the linked list. We can speed up the solution somewhat by making the pointer move forward by 2/4/8, and if we find that the value of the current node is smaller than the value of the previous node we looked at, we know that the minimum lies between these two and we need to linearly search this range. However, this does not bring down the overall complexity which remains at O(n). I leave it to the reader to write the pseudo code for the solution that incorporates this pointer-jumping enhancement.

How can we further improve the solution?

As you may have noticed by now, we are told that the list of numbers we have is strictly increasing until we reach the end of the list, at which point it wraps over, and we again have a strictly increasing list of numbers from that point. This circular list can be viewed as two list segments, each containing the sorted list of numbers—the first segment starting from the arbitrary pointer we have been given to the end of the linked list, and the second segment starting from the true beginning of the list to the element just before the arbitrary pointer we have been given. Let us refer to these segments as SEG1 and SEG2. This helps us mull over these segments easily. If SEG2 has zero length, what does it mean?

It means that the arbitrary pointer we have been given is in fact the true beginning of the sorted list and, hence, the minimum is pointed to the pointer 'p' we have been given. If SEG2 has non-zero length, we need to determine the start of SEG2, since that would point to the true beginning of the list and hence will contain the minimum. I leave it to the reader to consider the case when SEG1 has zero length.

Can we reduce the time complexity by employing some form of binary search on the circular linked list? As the reader can see, this is not possible, since binary search requires random access, and given that we have a circular linked list, we cannot use binary search to speed this up. This is a typical example of a case, where due to the limitation of the underlying data structure (the circular linked list, in this case), we cannot employ an algorithm to speed up the solution. Hence, linear search remains the best solution for this case.

Now, if we consider a variant of the above problem wherein we lift the restriction that the list is in fact represented by a circular singly linked list, and allow the list to be represented by a circular doubly linked list, can we do any better? It is easy to show that since a linked list does not support random access, using a doubly linked list does not bring down the asymptotic complexity from O(n). Now let us assume that we relax the constraints even further and allow an array to represent the circular list, can we use binary search to improve the O(n) solution we have achieved? I leave this question to the reader to answer.

This month's takeaway problem

This month's takeaway problem comes from graph theory. Given a directed graph, a Strongly Connected Component (SCC) is a set of nodes such that every pair of nodes in the SCC is mutually reachable from another. In simple terms, SCC corresponds to reducible loops in the graph. A directed graph can contain multiple strongly connected components. One of the interesting applications of SCCs is to represent the loops in the software programs we write. For example, a *for* loop you write in your 'C' code is internally represented as a SCC when the compiler analyses the function for optimisation opportunities. Can you come up with an algorithm to find all the strongly connected components of the graph?

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me. Feel free to send me your solutions and feedback to sandyasm_AT_yahoo_DOT_com. Till next month, happy programming! END

About the author:

Sandya Mannarswamy. The author is a specialist in compiler optimisation and works at Hewlett-Packard India. She has a number of publications and patents to her credit, and her areas of interest include virtualisation technologies and software development tools.